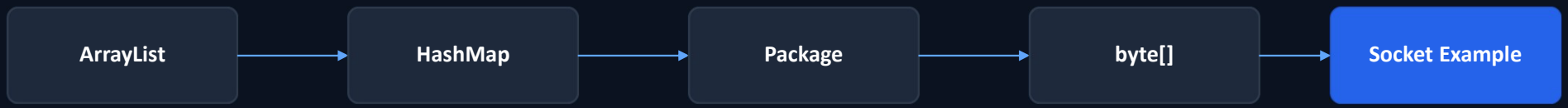
บทที่ 3

Java Collections & Data Handling

เตรียมผู้เรียนให้พร้อมอ่านโค้ด HLA ที่มี Handle, Map, byte



ภาพรวม Day 3

จากข้อมูลหลายรายการ → จัดโค้ดเป็นระบบ → เชื่อมแนวคิดไป HLA

1	**Collections**	ArrayList / HashMap / For-Each
2	**Data Handling**	Object ใน Collection, ค้นหา, update, loop
3	**Code Structure**	Package, Import, แยก Class ตามหน้าที่
4	**HLA Prep**	Handle→Value, byte[], Callback data
5	**Mini Project**	Package Tracking System

ทำไม Day 3 ต้องเรียน Collections?

HLA ไม่ได้ส่งข้อมูลแค่ตัวแปรเดี่ยว แต่เจอชุดข้อมูลและ mapping ตลอดเวลา

ตัวแปรเดี่ยว

```
String name = "PK001";
int count = 1;
```

เหมาะกับข้อมูลจำนวนน้อย
แต่ไม่พอสำหรับระบบจริง

ข้อมูลหลายรายการ

```
ArrayList<PackageData>
HashMap<String, PackageData>
```

เก็บหลาย Object
ค้นหาและวนลูปได้

แนวคิดที่ต่อเข้า HLA

```
ParameterHandleValueMap
AttributeHandleValueMap
byte[] tag
```

HLA ใช้ pattern แบบ
Handle → Value บ่อยมาก

ArrayList: กล่องเก็บข้อมูลหลายรายการ

ใช้เมื่อจำนวนข้อมูลเปลี่ยนได้ และต้องการวนอ่านทีละรายการ

```java
import java.util.ArrayList;
```

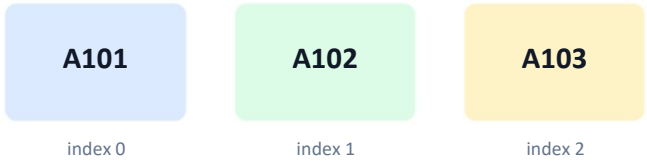
```java
    public static void main(String[] args) {
        ArrayList<String> rooms = new ArrayList<>();


        rooms.add("A101");
        rooms.add("A102");
        rooms.add("A103");


        System.out.println(rooms.get(1)); //A102


    }
```

จำภาพง่าย ๆ



- เพิ่มข้อมูลด้วย add()
- อ่านตำแหน่งด้วย get(index)
- นับจำนวนด้วย size()
- ลบข้อมูลด้วย remove()

HashMap: ค้นหาด้วย Key

เหมาะมากกับงานที่มีรหัสอ้างอิง เช่น TrackingNumber หรือ Handle

```
import java.util.HashMap;
```

```
public class HelloWorld {

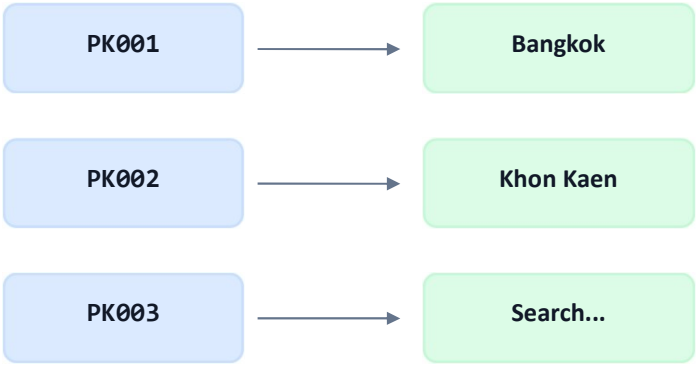

    public static void main(String[] args) {
        HashMap<String, String> locationMap = new HashMap<>();

        locationMap.put("PK001", "Bangkok");
        locationMap.put("PK002", "Khon Kaen");


        String location = locationMap.get("PK001");
        System.out.println(location);

    }
}
```

Key → Value



ใน HLA จะเจอแนวคิดคล้ายกัน:
Handle เป็นตัวระบุ → Value เป็นข้อมูลจริง

For-Each & Collection Methods

อ่านข้อมูลทั้งหมดใน Collection โดยไม่ต้องจัดการ index เอง

```
for (String room : rooms) {
    System.out.println(room);
}


for (PackageData pkg : packageList) {
    pkg.printInfo();
}
```

Method ที่ใช้บ่อย

- add(value) เพิ่มข้อมูล
- get(index) อ่านข้อมูลตามตำแหน่ง
- put(key, value) เพิ่มข้อมูลแบบ Map
- get(key) ค้นหาด้วย key
- containsKey(key) เช็คว่ามี key หรือไม่

Concept สำคัญ: Collection ไม่ได้เก็บแค่ int/String แต่สามารถเก็บ Object ที่เราสร้างเองได้ เช่น PackageData, Student, Room, FederateInfo

Code Structuring: Package & Import

พอโปรแกรมเริ่มใหญ่ ต้องแยกไฟล์ตามหน้าที่ ไม่เขียนทุกอย่างใน Main

โครงสร้างไฟล์

```
src/
 └─ workshop/
     ├─ Main.java
     ├─ PackageData.java
     └─ PackageManager.java
```

Package

```
package workshop;

public class PackageData {
    ...
}
```

Import

```
import java.util.HashMap;
import workshop.PackageData;

public class Main {
    ...
}
```

byte[] เบื้องต้น: ข้อมูลดิบก่อนแปลงเป็นค่าจริง

HLA มักส่งข้อมูลในรูปแบบ encoded data จึงควรรู้จัก byte[] ไว้ก่อน

```
byte[] tag = { 72, 76, 65 };

String text = new String(tag);
System.out.println(text); // HLA
```

มองเป็นกล่องเลข 8-bit



byte[] คือข้อมูลดิบที่ต้อง encode/decode ก่อนใช้งานจริง โดยเฉพาะตอนเข้า HLA Callback

Argument Passing over Network

ส่งข้อมูลจากโปรแกรมหนึ่งไปยังอีกโปรแกรมหนึ่งผ่าน TCP/IP Socket

Client A

เปิด Port และรอรับการเชื่อมต่อ

```
ServerSocket serverSocket = new
ServerSocket(5000);

Socket socket = serverSocket.accept();

String msg = input.readUTF();
```

รับข้อมูล

TCP/IP Socket

127.0.0.1 : 5000

1 ต่อ 1

Client B

Connect ไปยัง Client A แล้วส่งข้อมูล

```
Socket socket = new Socket("127.0.0.1",5000);

String msg = "hello world from ClientB";

output.writeUTF(msg);
```

ส่งข้อมูล

แนวคิดสำคัญ: ตัวแปร Java ไม่ได้ “วิ่งข้ามเครื่อง” โดยตรง แต่ค่าจะถูกเขียนลง Stream แล้วส่งผ่าน Socket ไปให้อีกฝั่งอ่านกลับมา

TCP/IP Socket Example

ตัวอย่าง 1-to-1: Client B ส่งข้อความให้ Client A

Client A — Receive

```java
package network;


import java.io.DataInputStream;
import java.net.ServerSocket;
import java.net.Socket;


public class ClientA {
    public static void main(String[] args) {
        int port = 5000;

        try {
            ServerSocket serverSocket = new ServerSocket(port);
            System.out.println("[A] waiting..");
            Socket socket = serverSocket.accept();
            System.out.println("[A] connected..");
            DataInputStream input = new DataInputStream(socket.getInputStream());
            String msg = input.readUTF();
            System.out.println("[A] Received: " + msg);


            input.close();
            socket.close();
            serverSocket.close();


        } catch (Exception e) {
            e.printStackTrace();
        }
    }

}
```

readUTF()

Client B — Send

```java
package network;


import java.io.DataOutputStream;
import java.net.Socket;


public class ClientB {
    public static void main(String[] args) {
        String aIp = "127.0.0.1";
        int port = 5000;


        try {
            Socket socket = new Socket(aIp, port);
            DataOutputStream output = new DataOutputStream(socket.getOutputStream());
            String msg = "hello world from ClientB";
            output.writeUTF(msg);
            output.flush();

            output.close();
            socket.close();

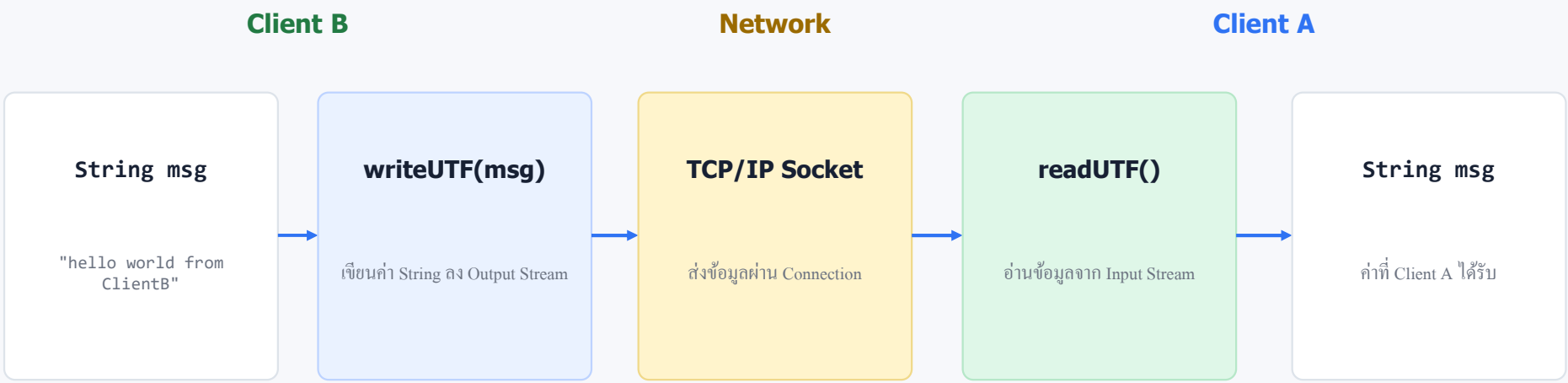
        } catch (Exception e) {
            e.printStackTrace();
        }
    }


}
```

writeUTF()

Data Flow: Client B → Client A

มองให้ออกว่า “ข้อมูล” เปลี่ยนผ่านจุดใดบ้าง



ฝั่งส่ง `output.writeUTF(msg);`

นำค่าจากตัวแปร msg เขียนลง Stream

ฝั่งรับ `String msg = input.readUTF();`

อ่านค่าจาก Stream แล้วสร้างตัวแปร msg ของ Client A

```
[A] Received: hello world from ClientB
```